

Google Cloud Spanner

Table of Contents

1. Introduction

Google Cloud Spanner is a managed relational database service designed for applications that need relational database capabilities together with strong consistency, high availability and large-scale growth. This guide is written for developers, data engineers and new interns. Each topic follows the same learning pattern: what it is, why it is used, when to use it, how it works or is configured, and a practical example or scenario.

Learning approach: Do not learn Spanner as a list of console fields. Learn the database architecture first, then connect each console setting to the problem it solves.

2. What is Cloud Spanner?

Spanner is a managed relational database on Google Cloud. It provides familiar relational concepts such as tables, rows, columns, SQL, primary keys and transactions, while being designed for workloads that need to scale and remain highly available.

What Spanner provides

Scenario: An e-commerce system may have many users creating orders at the same time. Order records, payment state and inventory changes may need transactional correctness while the application continues to grow.

3. Why do we need Spanner?

A traditional relational database is often a good choice. Spanner becomes interesting when the application combines relational transactions with requirements for scale, availability or distributed deployment.

Important: Spanner is not automatically the right database for every project. Compare the application's scale, latency, consistency, availability, workload type and cost before selecting it.

4. Spanner architecture and hierarchy

Before using the console, understand the hierarchy that you will see throughout the UI and documentation.

Google Cloud Project ↓ Spanner Instance ↓ Database ↓ Tables ↓ Rows + Columns

Simple mental model: The instance is the database environment. The database contains the application's schema and data. Tables organize records, while rows and columns hold the actual data.

5. Spanner console: Instances page

The Instances page is usually the starting point when administering Spanner. It shows the instances available in the selected project and exposes the actions used to create and manage them.

Figure 1 — Spanner Instances page from the supplied KT environment.

What the user should look for

How to use this screen: Start by identifying the correct project, then locate the instance you need. Before changing compute or configuration, understand what workload the instance serves.

6. Creating a Spanner instance

An instance is the foundation of a Spanner deployment. It defines the environment in which databases run, including location/replication configuration and compute capacity.

How to create an instance in the UI

Open the Google Cloud Console and go to Spanner.

Select the required Google Cloud project.

Open Instances and click Create instance.

Choose the instance edition that fits the workload.

Provide a clear instance display name and instance ID.

Choose the instance configuration/location.

Choose the initial compute capacity.

Select the scaling method available for the selected setup.

Review the configuration and estimated cost.

Create the instance.

Choosing the configuration

Decision point: Do not select the nearest region blindly. Consider user location, application components, latency, data residency, availability and cost together.

7. Processing units, nodes and storage

Spanner compute capacity is expressed using Processing Units (PUs) or nodes. In the supplied example, 1,000 PUs is shown as 1 node. Compute capacity and storage are different resources.

Common confusion: Adding compute does not mean you have added storage. Compute handles database operations; storage holds the data.

8. Instance Overview

After creating an instance, selecting it opens an Overview page. This view is useful for understanding the instance configuration, current capacity, utilization and databases.

Figure 2 — Instance Overview page from the supplied KT environment.

How to use the Overview: Use it as your first health check. Look at configuration, compute, CPU, throughput and storage before diving into more specialized troubleshooting tools.

9. Creating a database

A database is created inside an instance. It contains the schema, tables, indexes and application data. The instance provides the surrounding configuration and compute environment; the database contains the logical application data.

Typical UI workflow

Open the required Spanner instance.

Choose the option to create a database.

Provide the database name.

Choose the SQL dialect required by the application.

Define the initial schema or create the database and apply schema afterward.

Review the settings and create the database.

Example: The supplied KT environment contains an instance named ecom-core-instance with a database named ecom-analytics-db.

10. Database Overview

The Database Overview page moves from the instance level to the application-data level. It shows database details, schema-related information, performance data and tables.

Figure 3 — Database Overview page for ecom-analytics-db from the supplied KT environment.

11. Tables, rows, columns and indexes

Once the database exists, application data is organized into tables. Each table contains rows, and each row contains values for the table's columns.

Why indexes matter: An index can make a query faster, but it also introduces maintenance overhead. Create indexes based on real query patterns, not simply because an indexed column seems useful.

12. Primary key design

Primary keys do more than uniquely identify rows. In a distributed database, key design also affects how data is distributed and accessed.

Consider the sample OrdersLineItems table:

OrderId	LineItemId	ProductId	Quantity
1001	1	P101	2
1001	2	P205	1
1002	1	P101	3

A possible primary key is (OrderId, LineItemId). It uniquely identifies each line item.

Why should we think carefully about the key?

A key that receives a large concentration of reads or writes can create uneven access patterns. This is commonly discussed as a hotspot problem.

Practical rule: Choose a key based on uniqueness and how the application actually reads and writes the data. Do not choose a key only because it looks unique.

13. E-commerce data model example

A simple e-commerce database may use several related tables. The purpose is to show how the relational model is used in a realistic application.

Customer places order ↓ Create order record ↓ Insert order line items ↓ Reduce inventory ↓ Record payment status

Why transactions matter here: Related operations may need to succeed together. A partial update can leave the system in an inconsistent state.

14. Transactions and consistency

A transaction groups related database operations into one logical unit. Spanner supports ACID transactions, which help applications preserve correctness when multiple changes belong together.

Example

Suppose a customer places an order for the last item in stock. The application may need to create the order and reduce inventory as one logical transaction. If the inventory update fails, the application should not leave an order that claims success while inventory still shows the old quantity.

15. Replication and availability

Spanner maintains replicated copies of data according to the selected instance configuration. Replication is part of the architecture used to provide availability and resilience.

For a beginner, the important distinction is that you normally do not manually copy tables to another database server. The service manages replication according to the selected configuration.

Scenario: A production application may require resilience to a location failure. The instance configuration becomes an architectural decision rather than just a console preference.

16. Backup and restore

Backups protect against accidental deletion or other situations where the application needs to recover from an earlier state. Backups and replication solve different problems.

Do not confuse them: Replication supports availability and resilience during normal operation. Backups support recovery from earlier data states. One is not a replacement for the other.

17. Change Streams

Change Streams capture changes made to data in a Spanner database. They are useful when another system needs to react to database changes.

Spanner ■ ■ data changes ▼ Change Stream ■ ▼ Downstream pipeline ■ ■ ■ ■ Analytics ■ ■ ■ Notifications ■ ■ ■
Another application

Why use Change Streams?

A normal query asks what data exists now. A Change Stream is used to observe changes as they occur, making it useful for event-driven downstream processing.

Scenario

A new order is inserted into Spanner. A Change Stream captures the change. A downstream Dataflow pipeline processes it and writes an analytics record to BigQuery.

18. Spanner console navigation

The console contains instance-level and database-level tools. Learn them based on the problem you are trying to solve rather than memorizing every menu.

Instance-level tools

Database-level tools

19. SQL and schema management

Spanner supports relational database concepts and SQL. The supplied database uses Google Standard SQL.

```
CREATE TABLE OrdersLineItems ( OrderId STRING(36) NOT NULL, LineItemId INT64 NOT NULL, ProductId  
STRING(36), Quantity INT64 ) PRIMARY KEY (OrderId, LineItemId);
```

Why this matters: Schema design determines what data is stored and how it can be queried. Primary keys and indexes also affect access patterns and performance.

20. Performance troubleshooting

Performance issues should be diagnosed using the right tool. Looking only at CPU utilization can miss the real cause.

Troubleshooting approach

Start with the symptom: slow query, high CPU, high latency, contention or uneven traffic.

Identify whether the problem is query-related, transaction-related, locking-related or distribution-related.

Change one thing at a time and compare the result.

21. IAM and security

Production practice: Avoid broad project-level access when a narrower database or instance-level permission is sufficient.

22. When should we use Spanner?

23. Common beginner mistakes

Confusing compute capacity with storage.

Ignoring primary-key design.

Giving broad IAM permissions instead of least privilege.

Looking only at CPU when troubleshooting.

24. End-to-end example: e-commerce application

[illegible]

How the design works

The application performs related database operations.

The data is stored in relational tables.

A processing pipeline consumes those changes.

Analytics data is written to the analytical destination.

25. Creating a complete Spanner setup

Select the correct Google Cloud project.

Open Spanner and inspect the Instances page.

Create a development instance with an appropriate configuration and small starting capacity.

Open the instance Overview and identify its edition, region/configuration, compute capacity and storage.

Create a database inside the instance.

Open the database Overview.

Create or inspect a sample table such as OrdersLineItems.

Define a primary key based on application access patterns.

Run a few SQL queries in Spanner Studio.

Review backup settings.

Review the performance/insights menus so you know where to investigate problems later.

Learning outcome: The intern should be able to explain the difference between project, instance, database, table, primary key, compute capacity, backup and Change Streams without referring to notes.

27. Quick reference

Production-Ready Completion Guide

This section follows the same learning progression used in the Managed Airflow documentation: introduction → architecture → core concepts → UI/setup → configuration → security → monitoring → troubleshooting → performance/cost → reliability → production design → end-to-end scenario → common mistakes → operational checklist.

A. Standard learning path

34. Networking and connectivity

Spanner should be designed together with application networking, region/configuration and dependent services.

- Choose regional or multi-region configuration based on latency, availability, data residency and cost.
- Validate connectivity and private networking requirements.
- Document which services access Spanner and through which identities.

35. Cost and capacity planning

Compute capacity and storage are separate considerations. Increase capacity based on measured workload behavior.

- Track processing-unit/node usage and storage growth.
- Check query design, hotspots and lock contention before adding capacity.
- Review instance configuration against workload requirements.

36. Production monitoring and alerting

Use Spanner insights and monitoring as an operational workflow rather than checking CPU alone.

- Monitor utilization, storage, latency and throughput.
- Use Query, Lock, Hotspot and System Insights for diagnosis.

- Define alerts for sustained abnormal behavior.

37. Schema changes and safe operations

Treat schema, index and primary-key changes as production changes.

- Test changes in non-production.
- Review indexes against real query patterns.
- Document rollback/recovery considerations.

38. Reliability and recovery testing

Replication and backups solve different problems; production readiness requires tested recovery.

- Verify backup schedules and retention.
- Perform restore testing.
- Ensure Change Stream consumers handle expected event semantics.

39. Common production mistakes

- Poor primary keys causing hotspots.
- Treating backup as replication.
- Broad IAM instead of least privilege.
- Changing schema/indexes without query validation.
- Ignoring storage growth and capacity planning.

40. Operational checklist

- Correct project, instance and database selected.
- Region/replication matches requirements.
- Capacity and storage monitored.
- Least-privilege IAM.
- Backup and restore documented/tested.
- Insights tools used for troubleshooting.
- Primary-key/index design reviewed.
- CDC consumers monitored.
- Cost expectations documented.
- Production changes have an owner and recovery plan.

Official references

<https://docs.cloud.google.com/spanner/docs>

<https://docs.cloud.google.com/spanner/docs/monitoring-overview>

<https://docs.cloud.google.com/spanner/docs/change-streams>

Final learning outcome

After completing this document, the learner should be able to explain Cloud Spanner, create or configure its main resources, operate it from the Google Cloud console, troubleshoot common failures, and describe production controls.

Reference Tables

Section	Page
1. Introduction	3
2. What is Cloud Spanner?	3
3. Why do we need Spanner?	3
4. Spanner architecture and hierarchy	4
5. Spanner console: Instances page	4
6. Creating a Spanner instance	6
7. Processing units, nodes and storage	6
8. Instance Overview	7
9. Creating a database	8
10. Database Overview	8
11. Tables, rows, columns and indexes	9
12. Primary key design	9
13. E-commerce data model example	10
14. Transactions and consistency	10
15. Replication and availability	11
16. Backup and restore	11
17. Change Streams	11
18. Spanner console navigation	12
19. SQL and schema management	13
20. Performance troubleshooting	13
21. IAM and security	14
22. When should we use Spanner?	14
23. Common beginner mistakes	14
24. End-to-end example: e-commerce application	14
25. Creating a complete Spanner setup — beginner walkthrough	15
27. Quick reference	16

Requirement	How Spanner addresses it
Relational data	Tables, rows, columns, SQL, primary keys and indexes.
Correctness	ACID transactions and strong consistency.
High availability	Replicated data according to the selected instance configuration.
Scale	Compute capacity can be increased as workload grows.
Managed service	Google manages much of the underlying database infrastructure.

Situation	Why Spanner may help
Many users writing at the same time	Compute can be distributed across available capacity.

Situation	Why Spanner may help
Application cannot easily tolerate downtime	Spanner is designed for high availability.
Transactions must remain correct	Spanner supports ACID transactions.
Data and traffic keep growing	Compute capacity can be scaled.
Application needs a relational model	Spanner provides SQL, tables, keys and indexes.

Level	What it represents	Example
Google Cloud project	Container for cloud resources.	Selected GCP project.
Instance	Spanner environment containing configuration and compute capacity.	
Database	Logical database inside an instance.	ecom-analytics-db
Schema / Tables	Structure used to organize application data.	OrdersLineItems
Rows	Individual stored records.	One order line item
Columns	Fields belonging to a row.	OrderId, ProductId, Quantity

Console item	What it means	Why it matters
Create instance	Starts instance creation.	An instance is required before creating a database.
Instance name	Friendly name shown in the UI.	Helps identify the environment.
ID	Unique instance identifier.	Used by APIs and commands.
Edition	Selected Spanner edition.	Affects available capabilities and pricing/features.
Configuration	Location and replication arrangement.	Affects latency, availability and data placement.
Processing units	Allocated compute capacity.	Determines available database processing capacity.
Nodes	Larger compute unit.	1 node represents 1,000 processing units in the example.
Scaling mode	How compute is allocated.	Manual or available autoscaling configuration.
Storage utilization	Current storage usage view.	Helps monitor storage growth.

Configuration	Use case
Regional	Application and data can remain within one region.
Dual-region	Higher availability and regional resilience where supported.
Multi-region	Globally distributed applications or stronger regional resilience.

Term	Meaning	Example
Processing Unit (PU)	Small unit used to express compute capacity.	1,000 PUs
Node	1,000 processing units.	1 node = 1,000 PUs
Compute capacity	Resources available to process database operations.	1 node in the example
Storage	Space occupied by the databases and their data.	Measured independently from compute

Field / section	What it tells you
Edition type	Spanner edition used by the instance.
Instance ID	Unique instance identifier.
Configuration	Location and replication arrangement.

Field / section	What it tells you
Scaling mode	How compute capacity is allocated.
Default backup schedule	Whether a default backup schedule is configured.
Data replication to BigQuery	Whether a replication stream has been configured.
Compute capacity	Current allocated processing capacity.
CPU utilization	How much available compute is being used.
Operations	Read/write operation activity.
Throughput	Data being read/written.
Total database storage	Storage currently used by databases.

Item	Why it matters
Database name	Identifies the logical database.
Database dialect	Determines SQL syntax used by the database.
Encryption type	Shows how the database data is encrypted.
Scheduled backups	Shows the configured backup schedule.
Schema updates	Shows schema-change status.
CPU utilization	Helps understand compute pressure.
Operations	Shows read/write activity.
Throughput	Shows data movement rate.
Total database storage	Shows storage consumed by this database.

Concept	Example	Purpose
Table	OrdersLineItems	Stores order line-item records.
Row	One order line item	Represents one record.
Column	OrderId	One attribute of the record.
Primary key	(OrderId, LineItemId)	Uniquely identifies a row.
Index	An additional access path	Helps queries find data efficiently.

Table	Purpose
Customers	Customer details.
Orders	Order-level information.
OrdersLineItems	Products included in each order.
Products	Product information.
Payments	Payment transaction information.
Inventory	Available product quantity.

ACID property	Beginner explanation
Atomicity	All required changes happen together or the transaction is rolled back.
Consistency	The database stays within its defined rules.

ACID property	Beginner explanation
Isolation	Concurrent transactions are handled so they do not incorrectly interfere.
Durability	Committed data is stored reliably.

Feature	Purpose
Backup schedule	Creates backups according to a configured schedule.
Restore	Creates a database from a suitable backup when recovery is required.
Backup retention	Determines how long backups are retained.

Menu	Use it for
Overview	Configuration, capacity, utilization and databases.
Import/Export	Move data using supported workflows.
Backup/Restore	Create, manage and restore backups.
Instance partitions	View/manage partition capabilities where applicable.
System insights	Broader system performance analysis.
Query insights	Investigate expensive or slow queries.
Lock insights	Investigate lock contention.
Transaction insights	Investigate transaction performance.
Hotspot insights	Identify uneven access patterns.

Menu	Use it for
Overview	Database details, metrics and tables.
Spanner Studio	Run SQL and work with the database.
Import/Export	Move data using supported workflows.
Backup/Restore	Manage database backups.
Operations	View database/schema operations.
Change streams	Capture data changes for downstream processing.
System insights	Monitor database/system behavior.
Query insights	Investigate query performance.
Lock insights	Investigate locking and contention.
Transaction insights	Investigate transaction performance.
Hotspot insights	Find uneven access patterns.
Key Visualizer	Visualize key-range access patterns.

Tool	Use it when...
Query Insights	A query is slow or unexpectedly expensive.
Lock Insights	Transactions are waiting because of contention.
Transaction Insights	Transactions have latency or performance problems.
Hotspot Insights	One part of the key space receives much more traffic.

Tool	Use it when...
Key Visualizer	You need a visual view of key-range access.
System Insights	You want a broader view of system/database performance.

Concept	Meaning
IAM principal	User, group or service account receiving access.
Role	Collection of permissions.
Least privilege	Give only the access required for the task.
Service account	Identity used by an application or cloud service.
Encryption	Spanner encrypts stored data and the console shows the encryption configuration.

Use Spanner when...	Think twice when...
Relational transactions plus large scale are required.	The workload is small and simple.
High availability is important.	A simpler managed relational database already meets the requirements.
The workload can grow significantly.	The workload is mainly object/file storage.
The application benefits from distributed database architecture.	The workload is primarily analytical scanning better suited to a warehouse.

When you need to...	Start by thinking about...
Understand the deployment hierarchy	Project → Instance → Database → Tables
Increase processing capacity	Processing units / nodes
Increase resilience	Instance configuration and replication
Recover earlier data	Backups and restore
Keep relational correctness	Transactions and ACID
Improve query performance	Indexes + Query Insights
Investigate lock contention	Lock Insights
Investigate uneven traffic	Hotspot Insights / Key Visualizer
React to database changes	Change Streams
Control access	IAM + least privilege

Stage	Expected outcome
1. Introduction	Explain the service, problem and role.
2. Architecture	Explain components and flow.
3. Core concepts	Explain hierarchy, execution and terminology.
4. UI and setup	Create resources and recognize key settings.
5. Configuration	Configure runtime, identity, network and scaling.
6. Security	Apply IAM, encryption, secrets and least privilege.
7. Monitoring	Use logs, metrics and alerts.
8. Troubleshooting	Move from symptom to evidence to root cause.
9. Performance and cost	Relate design to throughput, latency and cost.

Stage	Expected outcome
10. Reliability	Handle retries, duplicates and recovery.
11. Production design	Apply CI/CD, environment separation and ownership.
12. End-to-end scenario	Explain the service in a realistic pipeline.